Does the learning matter? A pre-registered deconstruction of SARSA-guided information exchange in the MPHBS hybrid metaheuristic

Abinaya G[a,*]

[a] *[PENDING: department, institution], [PENDING: city], [PENDING: country]*

Abstract

MPHBS couples the Honey Badger Algorithm and the Marine Predators Algorithm through a mediator that assembles hybrid solutions dimension by dimension under the control of a SARSA-style Q-table. Its original evaluation attributes part of the gains to this learning, mainly through a comparison with a "matched" random mirror. We show that this mirror is not matched per iteration: it re-evaluates the MPA population in every iteration, so it runs 21% (CEC2022) and 16% (FIR) fewer iterations at the same evaluation budget, and on CEC2022 it also refreshes a population memory that MPHBS leaves unchanged after the mediator. We then report a pre-registered study with budget-identical controls on CEC2022 ($D = 20$) and eight FIR filter-design problems ($D = 31$): 14,300 seed-paired runs executed once, analysed with a frozen script. A random mediator at the same budget performs at least as well as SARSA on both benchmarks (mean paired win probability of SARSA 0.461, 95% CI [0.434, 0.489], and 0.443, [0.409, 0.476]). The gains of MPHBS come instead from the exchange step itself (decisive on FIR) and from the restructured main loop, which outperforms the original backbone on all 20 problems even without a mediator. Refreshing the memory, as the published mirror does, is strongly harmful on CEC2022 (exploratory). We conclude with recommendations for designing ablation controls for learned components in metaheuristics.

Keywords: Hybrid metaheuristics, Reinforcement learning, Ablation, Experimental methodology, Pre-registration, Reproducibility

1. Introduction

Learned control components are now common in metaheuristics: reinforcement learning chooses operators, sets parameters, or decides how solution information flows between cooperating populations. Whether the *learning* is responsible for the reported gains is usually argued with an ablation in which the learned controller is replaced by a random one. Such an ablation is informative only if the two variants differ in nothing but the learning. When the controlled variant also changes how evaluations are spent, how many iterations are run, or how the populations' internal state evolves, the comparison can attribute to learning an effect that has a different cause.

This paper examines one recent instance in detail. MPHBS [1] hybridises the Honey Badger Algorithm (HBA) [2] and the Marine Predators Algorithm (MPA) [3] through a mediator that assembles hybrid vectors from ranked subsets of both populations, guided by a SARSA-style value table [4]. The authors compare SARSA with a "matched random mirror" and report that SARSA is better on CEC2022 [5] but not on their FIR filter-design problems. Their code and per-run results are public [6], which makes a precise re-examination possible.

Our contributions are:

1. We show from the authors' code that the published random mirror spends N additional evaluations per iteration and, under the CEC2022 configuration, additionally refreshes the MPA population memory after the mediator, which MPHBS does not do (Section 3). Our port of the mirror agrees with the authors' published runs on all 12 CEC2022 functions and all 8 FIR cases.
2. We report a pre-registered study [7] with budget-identical controls that separate four candidate causes of MPHBS's performance: the learning, the evaluation budget of the control, the memory refresh, and the restructured main loop (Section 4).
3. We find no benefit of learning at equal budget, a decisive benefit of the exchange step on FIR, and a large benefit of the loop restructuring on both benchmarks (Section 5).
4. We derive practical requirements for ablation controls of learned components (Section 6).

The study continues a line of work that deconstructs metaheuristics to identify which components actually carry their performance [8, 9, 10, 11, 12]. Unlike that work, our object is not a metaphor but a learned control component, and our main instrument is a controlled, pre-registered ablation. [PENDING: literature check: confirm that no published work re-examines MPHBS or learned information-exchange mediators in this way; add the closest related studies here.]

*Corresponding author.
Email address: PENDING-email@example.org (Abinaya G)

2. MPHBS and its published evidence

2.1. Algorithm

MPHBS evolves an HBA population and an MPA population of N agents each. Every iteration consists of (i) the native HBA and MPA moves with greedy replacement and MPA's memory saving, in which an agent whose new position is worse than its stored memory reverts to it; (ii) a FAD perturbation stage of MPA, evaluated in place with greedy acceptance, placed before (CEC2022) or after (FIR) the mediator; and (iii) the mediator (Phase 2). The mediator draws ranked subsets of $N_{\text{sub}} = \text{round}(\rho N)$ agents from each population, starts from the global best, and runs $E = N_{\text{sub}} N_i$ episodes. In each episode a block of dimensions is reassigned: K candidate vectors are assembled by taking each active dimension from a (source, rank) pair chosen by one of several policies over the Q-table, a cheap proxy score selects one candidate, and only that candidate is evaluated. The reward is the sign of the improvement, worse candidates may be accepted with a small, decreasing probability, and the Q-table is updated by a SARSA rule. Accepted candidates replace the worst members of the subsets, which are written back to the populations at the end of Phase 2. With $N = 30$ the final configurations are B-C1 on CEC2022 ($K = 7$, $N_i = 3$, $\rho = 0.25$, so $N_{\text{sub}} = 8$ and $E = 24$) and A-C1 on FIR ($K = 3$, $N_i = 9$, $E = 72$), selected by the authors on the same benchmarks [1].

2.2. Evidence for the role of learning

The authors evaluate a random mirror that, according to its description, keeps the backbone, candidate memories, proxy screening, acceptance rule and FAD placement, and only replaces the Q-table by uniform random choices of source and rank. Re-analysing their published 30 common-seed runs with paired Wilcoxon tests [13] and Holm correction [14] within each benchmark, SARSA is significantly better than the mirror on 8 of 12 CEC2022 functions (worse on 1), and on 0 of 8 FIR cases. On CEC2022 F10, a bimodal composition function, SARSA escapes the dominant local basin (final error < 50) in 17/30 runs against 4/30 for the mirror and 0/30 for the backbone without mediator.

3. Why the published control is not matched

Reading `MPHBS_main.m` and `MPHBS_random_mirror.m` side by side reveals two differences that the description of the mirror does not mention (Fig. 1).

Evaluation budget per iteration. MPHBS spends $3N + 1 + E$ evaluations per iteration: $2N$ for the native moves, N for the evaluated FAD stage, one for re-evaluating the starting point of the mediator and E for the episodes. The mirror additionally re-evaluates the whole MPA population at the start of every iteration, i.e. $4N + 1 + E$. Both are run for the largest number of iterations that fits the total budget, so the mirror runs $T = 1378$ instead of 1738 iterations on CEC2022 (-21%) and 776 instead of 919 on FIR (-16%), and performs correspondingly fewer mediator episodes. The authors' own result files confirm this:

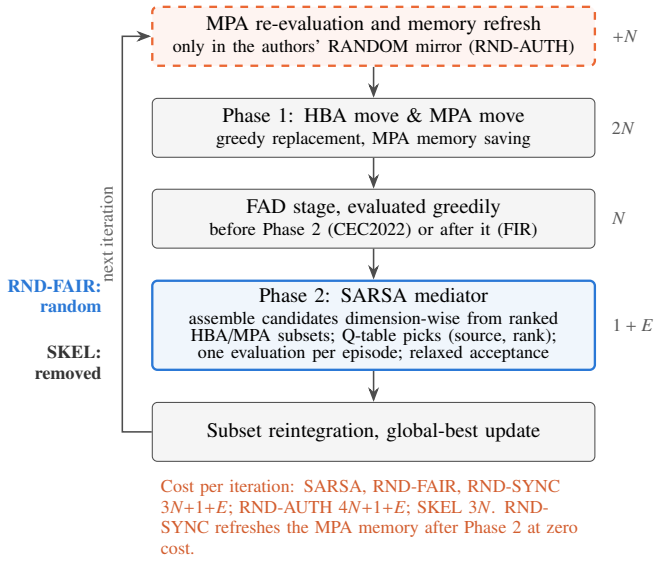


Figure 1: One MPHBS iteration with its evaluation cost (N agents per population, E mediator episodes). The dashed block exists only in the authors' random mirror. The arms of Table 1 replace the mediator's policy (RND-FAIR), remove Phase 2 (SKEL), or refresh the MPA memory after Phase 2 at no cost (RND-SYNC).

their mirror runs total 199,870 evaluations on CEC2022 and 149,828 on FIR, against 199,930 and 149,857 for MPHBS, exactly as these per-iteration costs predict.

Memory refresh. The re-evaluation returns values that are already known, but it is followed by MPA's memory-saving step, which sets the stored memory to the current population. Under the FIR configuration MPHBS performs the same update after its final FAD stage, so the extra block changes nothing except the budget. Under the CEC2022 configuration, however, MPHBS last updates the memory *before* the mediator and never after it: in the next iteration, MPA agents that move to a worse position revert to their pre-mediator positions. The mirror instead reverts them to the post-mediator population. On CEC2022 the mirror therefore differs from MPHBS in two ways at once.

Other differences. The mirror also omits the Q-table bookkeeping and the closest-action projection; once the Q-table is not used, these only set table indices and draw no random numbers, so they cannot affect the search. The global-best injection inside the mediator writes into subset rows that are overwritten at the end of Phase 2 in both codes; we verified with bit-identical re-runs that removing it has no effect (Section 4.5).

4. Study design

The design, hypotheses, run counts, statistics and decision rule were fixed in a pre-registration that was frozen and tagged in the public repository before any campaign run (Section 7). We summarise it here; no analysis reported as confirmatory deviates from it.

4.1. Arms

All arms are implemented on a validated Python port of the authors' code (Section 4.5) and use the authors' final configuration of each benchmark (Table 1). **SARSA** is MPHBS unchanged. **RND-FAIR** draws source and rank uniformly at random for all K candidates and never reads or updates a Q-table; everything else, including the evaluation cost, is identical to SARSA. **RND-SYNC** (CEC2022 only) is RND-FAIR plus the mirror's memory refresh, performed at zero evaluation cost because all fitness values are known; on FIR it would be identical to RND-FAIR. **RND-AUTH** is a faithful port of the authors' mirror, including the extra re-evaluation. **SKEL** is the MPHBS loop with Phase 2 removed. **MPHB** is the authors' backbone without mediator, whose loop differs from MPHBS's in that its FAD perturbation is not evaluated in place but replaces the population and is evaluated in the next iteration. Secondary arms vary the number of episodes (N_i), remove proxy screening ($K = 1$), make the acceptance greedy, or add the memory refresh to SARSA (SARSA-SYNC).

4.2. Hypotheses

Each hypothesis is a paired comparison of two arms (anchor first), tested separately on each benchmark: **H1** (learning) SARSA vs RND-FAIR; **H2a** (budget of the published mirror) RND-SYNC vs RND-AUTH on CEC2022 and RND-FAIR vs RND-AUTH on FIR, which differ only in the extra evaluations; **H2b** (memory refresh, CEC2022) RND-FAIR vs RND-SYNC; **H3** (mediator) SARSA vs SKEL; **H4** (loop) SKEL vs MPHB. H1–H4 are confirmatory. The escape rate on CEC2022 F10 (H5), the number of episodes (H6), the mediator's components (H7) and the memory refresh in SARSA (H8) are exploratory.

4.3. Benchmarks, runs and seeds

CEC2022 [5] F1–F12 at $D = 20$ with 200,000 evaluations (official C code), and the eight FIR filter-design cases of [1] (low-pass, high-pass, band-pass and band-stop, each in an easier and a harder variant) at $D = 31$ with 150,000 evaluations. The outcome is the final error (CEC2022) or objective value (FIR). The H1 pair was run 100 times per instance and every other arm 50 times; F10 was extended to 100 runs for the four H5 arms. Seeds are seed = $92\,000\,000 + \text{offset} + 1000 \cdot \text{instance} + \text{run}$, identical across arms, so all comparisons are seed-paired. The H1 run count was chosen so that a practical-equivalence claim has adequate power (pre-registration, Section 4).

4.4. Statistics and decision rule

For each instance we compute a two-sided paired Wilcoxon signed-rank test and the paired win probability $\hat{w}$ = (wins + 0.5 ties)/n of the anchor; values within 10^{-12} are ties. p-values are Holm-adjusted within each (hypothesis, benchmark) family [14, 15]. The domain-level effect $\bar{W}$ is the mean of $\hat{w}$ over instances, with a 95% percentile bootstrap interval [16] obtained by resampling runs within instances. The verdict is applied mechanically in this order: *mixed* if Holm-significant instances favour both arms; *anchor (comparator) superior* if at least one Holm-significant instance favours it and the interval for $\bar{W}$ lies entirely above (below) 0.5; *practically equivalent* if the interval lies within [0.44, 0.56], the negligible-effect band of Vargha and Delaney [17], and no instance is significant; otherwise *inconclusive*. Escape rates (H5) are reported with Wilson intervals [18] and compared by an exact McNemar test [19].

4.5. Execution and validation

The campaign ran once on GitHub-hosted Linux runners in the pinned software environment (Python 3.11, NumPy 2.4.6), after the full test suite had passed there. The port's MPHBS reproduces the authors' evaluation totals exactly and their published run distributions [6]; the one known exception, a lower escape rate on F10, is examined in H5. Unit tests establish that (i) the variant code with default settings is bit-identical to the validated MPHBS port; (ii) every arm's evaluation total matches Table 1; (iii) at a fixed number of iterations and on test problems, RND-AUTH produces trajectories bit-identical to RND-FAIR under the FIR configuration and to RND-SYNC under the CEC2022 configuration, so the re-evaluation itself has no effect other than its cost; and (iv) removing the writes of the global-best injection does not change any trajectory. All 14,300 runs completed with the expected evaluation totals, and re-running the analysis on the stored raw results on a second machine reproduced all verdicts, effect sizes and intervals. RND-AUTH agrees with the authors' published mirror runs on every instance (two-sided Mann–Whitney, Holm; 0 of 12 and 0 of 8 instances differ).

5. Results

Table 2 lists the confirmatory verdicts and Fig. 2 the per-instance effects.

5.1. H1: learning

At equal budget, SARSA does not outperform random assembly on either benchmark. On CEC2022 SARSA's mean paired win probability is 0.461 [0.434, 0.489], and on FIR 0.443 [0.409, 0.476]. Both intervals lie below 0.5 and one instance per benchmark is significant in favour of the random mediator (CEC2022 F3, FIR case 1; Table 3), so the pre-registered verdict is *comparator superior* on both. The effect is small: no instance favours SARSA, and the median errors of the two arms are close on most instances.

5.2. H2: the published mirror

With the memory refresh held fixed, the extra evaluations of the published mirror do not produce a conclusive difference on CEC2022 ($\bar{W} = 0.534$ [0.495, 0.573], verdict *inconclusive*). On FIR, where the re-evaluation affects only the budget, the budget-identical random mediator is better than the published mirror ($\bar{W} = 0.575$ [0.527, 0.623], *anchor superior*). The memory refresh, by contrast, has a large effect on CEC2022 (H2b): random assembly *without* the refresh beats the same mediator with it on 9 of 12 functions ($\bar{W} = 0.717$ [0.690, 0.743]), for instance on F5 with median errors of 16.53 and 417.6. Because 2 functions (F1, F11) favour the refresh,

Table 1: Primary arms. All arms use the authors' final configuration of the domain (B-C1 on CEC2022, A-C1 on FIR) and the same total budget (200,000 FE on CEC2022, 150,000 FE on FIR); the number of iterations T follows from the per-iteration cost. $N = 30$; $E = 24$ (CEC2022) and 72 (FIR) mediator episodes per iteration.

		FE per iteration		Iterations T	
Arm	Change relative to MPHBS	CEC2022	FIR	CEC2022	FIR
SARSA	none (reference: MPHBS)	115	163	1738	919
RND-FAIR	uniform random assembly, no Q-table	115	163	1738	919
RND-SYNC	RND-FAIR + zero-FE MPA memory refresh	115	–	1738	–
RND-AUTH	authors' RANDOM mirror (+N re-evaluations)	145	193	1378	776
SKEL	no Phase 2 (no mediator)	90	90	2221	1666
MPHB	authors' backbone without mediator	90	90	2221	1666

Table 2: Confirmatory contrasts (pre-registered H1–H4). Sig. (A/C): number of instances significant after Holm in favour of the anchor / comparator. $\bar{W}$: mean paired win probability of the anchor with 95% bootstrap CI. Verdicts by the frozen decision rule.

	Domain	Anchor	Comparator	n	Sig. (A/C)	$\bar{W}$	95% CI	Verdict
H1	CEC2022	SARSA	RND-FAIR	100	0/1	0.461	[0.434, 0.489]	comparator
H1	FIR	SARSA	RND-FAIR	100	0/1	0.443	[0.409, 0.476]	comparator
H2a	CEC2022	RND-SYNC	RND-AUTH	50	0/0	0.534	[0.495, 0.573]	inconclusive
H2a	FIR	RND-FAIR	RND-AUTH	50	1/0	0.575	[0.527, 0.623]	anchor
H2b	CEC2022	RND-FAIR	RND-SYNC	50	9/2	0.717	[0.690, 0.743]	mixed
H3	CEC2022	SARSA	SKEL	50	4/2	0.579	[0.545, 0.613]	mixed
H3	FIR	SARSA	SKEL	50	8/0	0.962	[0.943, 0.980]	anchor
H4	CEC2022	SKEL	MPHB	50	12/0	0.780	[0.757, 0.802]	anchor
H4	FIR	SKEL	MPHB	50	8/0	0.826	[0.798, 0.853]	anchor

Table 3: H1 per instance: SARSA vs RND-FAIR, 100 seed-paired runs each. Medians of the final error; W/T/L from SARSA's view; $\hat{w}$ = paired win probability of SARSA; p two-sided Wilcoxon, Holm-adjusted within domain. $^{\downarrow}$ significant in favour of RND-FAIR.

	Inst.	SARSA	RND-FAIR	W/T/L	$\hat{w}$	p_{Holm}
CEC2022	F1	2.43×10^{-7}	1.59×10^{-7}	36/0/64	0.36	0.635
CEC2022	F2	44.9	49.08	43/0/57	0.43	1
CEC2022	F3	1.63×10^{-5}	4.39×10^{-6}	21/0/79	$0.21^{\downarrow}$	1.0×10^{-7}
CEC2022	F4	37.31	33.83	38/0/62	0.38	0.180
CEC2022	F5	17.67	17.04	54/0/46	0.54	1
CEC2022	F6	20.02	17.65	51/0/49	0.51	1
CEC2022	F7	30.1	29.92	47/0/53	0.47	1
CEC2022	F8	20.96	21.01	46/0/54	0.46	1
CEC2022	F9	180.8	180.8	58/8/34	0.62	0.158
CEC2022	F10	100.4	100.4	52/0/48	0.52	1
CEC2022	F11	300	300	54/0/46	0.54	1
CEC2022	F12	240.1	240.8	49/0/51	0.49	1
FIR	Case 1	5.17×10^{-3}	4.81×10^{-3}	28/0/72	$0.28^{\downarrow}$	1.3×10^{-4}
FIR	Case 2	0.02158	0.02137	47/0/53	0.47	1
FIR	Case 3	5.09×10^{-3}	4.82×10^{-3}	39/0/61	0.39	0.181
FIR	Case 4	0.02151	0.02124	51/0/49	0.51	1
FIR	Case 5	1.74×10^{-3}	1.41×10^{-3}	36/0/64	0.36	0.335
FIR	Case 6	8.34×10^{-3}	6.77×10^{-3}	47/0/53	0.47	1
FIR	Case 7	1.32×10^{-3}	1.60×10^{-3}	52/0/48	0.52	1
FIR	Case 8	3.69×10^{-3}	3.81×10^{-3}	54/0/46	0.54	1

the verdict is *mixed*. Together, H2a and H2b explain the published SARSA-versus-mirror difference on CEC2022 without any contribution of learning: the published mirror carries a handicap that is mostly due to the memory refresh on CEC2022 and to the reduced budget on FIR.

5.3. H3 and H4: what does matter

Removing the mediator altogether is very costly on FIR: SARSA beats SKEL on all eight cases ($\bar{W} = 0.962$ [0.943, 0.980]). On CEC2022 the effect is mixed ($\bar{W} = 0.579$): the mediator helps on F1, F3, F9 and F10 and hurts on F5 and F7. The exchange of information between the two populations is therefore useful, but the evidence indicates that its value does not depend on learned choices of source and rank.

The restructured loop is itself a large source of improvement. SKEL, which has no mediator, beats MPHB on every CEC2022 function ($\bar{W} = 0.780$ [0.757, 0.802]) and every FIR case ($\bar{W} = 0.826$ [0.798, 0.853]). The two loops differ mainly in whether the FAD perturbation is evaluated with greedy acceptance; this change therefore accounts for part of the advantage of MPHBS over its backbone that a comparison of MPHBS with MPHB alone would attribute to the mediator.

5.4. Exploratory results (H5–H8)

F10 escapes. SARSA escapes the local basin of F10 in 26/100 runs (Wilson 95% interval [0.18, 0.35]) and RND-FAIR in 25/100 ([0.18, 0.34]; exact McNemar $p = 1.00$); RND-AUTH escapes in 10/100 and SKEL in 0/100. Escaping requires the mediator but not learning. The authors' 17/30 escapes for SARSA are inconsistent with our 100-seed estimate (binomial $p = 7.7\times10^{-10}$); since our port reproduces their other published distributions, we regard their single F10 sample as unrepresentative but cannot rule out a platform-dependent effect.

Mediator components and budget. Removing proxy screening (H7, $K = 1$) or relaxed acceptance leaves performance practically unchanged or inconclusive (Table 4). Fewer episodes per iteration are clearly worse on FIR ($N_i = 1$ vs 9: $\bar{W} = 0.345$

[0.300, 0.390]), whereas no tested N_i shows a consistent advantage of SARSA over random assembly (Table 4).

Memory refresh in SARSA. Adding the zero-cost refresh to SARSA reproduces the H2b pattern (H8, $\bar{W} = 0.277$ [0.252, 0.303]): the stale memory of MPHBS is beneficial on most CEC2022 functions. [PENDING: diversity diagnostic: population diversity and reversion counts per iteration for SARSA vs SARSA-SYNC, from the stored traces, to test the explanation below.]

6. Discussion

6.1. What drives MPHBS

Three components carry the performance of MPHBS on these benchmarks: exchanging information between the two populations at all, the amount of exchange per iteration, and the evaluated FAD stage of its loop. The SARSA controller is not among them: random choices of source and rank are at least as good. A plausible reason is the reward signal. It is the sign of the improvement of a single assembled vector over the current one, which says little about which (source, rank) pair helped, while the proxy screening and the acceptance rule already bias the search towards good candidates.

The memory finding suggests a mechanism. When MPHBS leaves the memory stale, MPA agents that move to worse positions revert to where they were before the mediator rather than to the post-mediator population, whose subset rows the mediator has filled with copies of vectors close to the incumbent. The stale memory may therefore preserve diversity. This explanation is exploratory and is tested by the pending diagnostic above.

6.2. Requirements for ablation controls of learned components

1. **Match the cost per iteration, not only the total budget.** When runs are cut by a total evaluation budget, any extra evaluation per iteration silently reduces the number of iterations of the control.
2. **Match every state update.** A control should be derived from the learned variant by removing the learning, not written as a separate program; side effects such as memory or archive updates must be identical.
3. **Verify equivalence mechanically.** With fixed seeds, the control with learning "switched off" in the same code path, and unit tests showing bit-identical trajectories where no difference is intended, rule out accidental differences.
4. **Separate the learned component from the mechanism it controls.** A no-mediator arm and a random-policy arm answer different questions (H3 vs H1); both are needed.
5. **Fix the analysis in advance.** Pre-registered hypotheses, run counts and decision rules prevent post hoc selection of the favourable comparison [7, 20, 21].

6.3. Threats to validity

Scope. We study one hybrid, two benchmark sets at one dimension each, and the authors' final configurations. Those configurations were selected with SARSA on the same benchmarks, which favours SARSA and makes H1 conservative. [PENDING: exploratory BBOB extension [22] to test whether H1 generalises.] **Port.** Our results use a Python port; agreement with the authors' MATLAB runs is distributional because random streams differ. The port reproduces the authors' totals and run distributions, and RND-AUTH reproduces their mirror on all instances. **Effect sizes.** The H1 effect is small; we claim the absence of a benefit of learning, not a large benefit of random choice. **Exploratory parts.** H5–H8 and the memory mechanism are exploratory and should be confirmed independently.

7. Conclusion

The learned controller of MPHBS does not improve on random choices at equal budget on CEC2022 or FIR; the published evidence for it rests on a control that is handicapped by its evaluation budget and, on CEC2022, by a memory update. MPHBS's advantage comes from exchanging information between its populations and from its restructured loop. More generally, ablations of learned components in metaheuristics should match the per-iteration cost and every state update of the learned variant, and should be verified mechanically and fixed in advance.

Data and code availability

Code, the frozen pre-registration (tag `pathA-prereg-frozen`), all 14,300 raw runs, the analysis and the scripts that generate every number, table and figure in this paper are available at `https://github.com/abinaya-g/la-kaie` (campaign commit `d5815d7`). [PENDING: archive a release on Zenodo and cite its DOI.]

CRediT authorship contribution statement

[PENDING: CRediT roles.]

Declaration of competing interest

[PENDING: declaration.]

Acknowledgements

We thank the authors of MPHBS for publishing their code and per-run results, without which this study would not have been possible. [PENDING: funding; whether the MPHBS authors were contacted before submission.]

Table 4: Exploratory contrasts (H6–H8), same statistics as Table 2; no claim rests on them.

	Domain	Anchor	Comparator	n	Sig. (A/C)	$\bar{W}$	95% CI	Verdict
H6 policy, N_i=1	CEC2022	SARSA (N_i=1)	RND (N_i=1)	50	0/0	0.465	[0.427, 0.504]	inconclusive
H6 policy, N_i=1	FIR	SARSA (N_i=1)	RND (N_i=1)	50	0/1	0.470	[0.422, 0.518]	inconclusive
H6 policy, N_i=3	CEC2022	SARSA	RND-FAIR	50	0/1	0.435	[0.396, 0.473]	comparator
H6 policy, N_i=3	FIR	SARSA (N_i=3)	RND (N_i=3)	50	0/0	0.435	[0.387, 0.483]	inconclusive
H6 policy, N_i=9	CEC2022	SARSA (N_i=9)	RND (N_i=9)	50	1/1	0.490	[0.450, 0.528]	mixed
H6 policy, N_i=9	FIR	SARSA	RND-FAIR	50	0/0	0.450	[0.403, 0.498]	inconclusive
H6 N_i=1 vs default	CEC2022	RND (N_i=1)	RND-FAIR	50	2/2	0.477	[0.441, 0.515]	mixed
H6 N_i=1 vs default	FIR	RND (N_i=1)	RND-FAIR	50	0/4	0.345	[0.300, 0.390]	comparator
H6 other N_i vs default	CEC2022	RND (N_i=9)	RND-FAIR	50	1/3	0.448	[0.411, 0.486]	mixed
H6 other N_i vs default	FIR	RND (N_i=3)	RND-FAIR	50	0/1	0.495	[0.448, 0.543]	inconclusive
H7 K=1	CEC2022	RND-K1	RND-FAIR	50	0/1	0.486	[0.448, 0.524]	inconclusive
H7 K=1	FIR	RND-K1	RND-FAIR	50	0/0	0.512	[0.465, 0.560]	equivalent
H7 greedy accept.	CEC2022	RND-GreedyAcc	RND-FAIR	50	0/0	0.507	[0.468, 0.547]	equivalent
H7 greedy accept.	FIR	RND-GreedyAcc	RND-FAIR	50	0/1	0.483	[0.435, 0.530]	inconclusive
H8	CEC2022	SARSA-SYNC	SARSA	50	2/9	0.277	[0.252, 0.303]	mixed

References

[1] E. Flores, R. Olivares, MPHBS: A novel hybrid metaheuristic with SARSA-guided information exchange, Knowledge-Based Systems 352 (2026) 117070.

[2] F. A. Hashim, E. H. Houssein, K. Hussain, M. S. Mabrouk, W. Al-Atabany, Honey badger algorithm: New metaheuristic algorithm for solving optimization problems, Mathematics and Computers in Simulation 192 (2022) 84–110.

[3] A. Faramarzi, M. Heidarinejad, S. Mirjalili, A. H. Gandomi, Marine predators algorithm: A nature-inspired metaheuristic, Expert Systems with Applications 152 (2020) 113377.

[4] R. S. Sutton, A. G. Barto, Reinforcement Learning: An Introduction, 2nd Edition, MIT Press, 2018.

[5] A. Kumar, K. V. Price, A. W. Mohamed, A. A. Hadi, P. N. Suganthan, Problem definitions and evaluation criteria for the CEC 2022 special session and competition on single objective bound constrained numerical optimization, Tech. rep., Nanyang Technological University (2021).

[6] E. Flores, R. Olivares, MPHBS v1.0.0 – reproducibility package for SARSA-guided HBA-MPA information exchange (2026). doi:10.5281/zenodo.21326361.

[7] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, D. T. Mellor, The preregistration revolution, Proceedings of the National Academy of Sciences 115 (11) (2018) 2600–2606.

[8] K. Sörensen, Metaheuristics—the metaphor exposed, International Transactions in Operational Research 22 (1) (2015) 3–18.

[9] C. L. Camacho-Villalón, M. Dorigo, T. Stützle, The intelligent water drops algorithm: why it cannot be considered a novel algorithm, Swarm Intelligence 13 (2019) 173–192.

[10] C. L. Camacho-Villalón, M. Dorigo, T. Stützle, An analysis of why cuckoo search does not bring any novel ideas to optimization, Computers & Operations Research 142 (2022) 105747.

[11] C. L. Camacho-Villalón, M. Dorigo, T. Stützle, Exposing the grey wolf, moth-flame, whale, firefly, bat, and antlion algorithms: six misleading optimization techniques inspired by bestial metaphors, International Transactions in Operational Research 30 (6) (2023) 2945–2971.

[12] C. Aranha, C. L. Camacho-Villalón, F. Campelo, M. Dorigo, R. Ruiz, M. Sevaux, K. Sörensen, T. Stützle, Metaphor-based metaheuristics, a call for action: the elephant in the room, Swarm Intelligence 16 (2022) 1–6.

[13] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1 (6) (1945) 80–83.

[14] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.

[15] J. Derrac, S. García, D. Molina, F. Herrera, A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms, Swarm and Evolutionary Computation 1 (1) (2011) 3–18.

[16] B. Efron, Bootstrap methods: another look at the jackknife, The Annals of Statistics 7 (1) (1979) 1–26.

[17] A. Vargha, H. D. Delaney, A critique and improvement of the CL common language effect size statistics of McGraw and Wong, Journal of Educational and Behavioral Statistics 25 (2) (2000) 101–132.

[18] E. B. Wilson, Probable inference, the law of succession, and statistical inference, Journal of the American Statistical Association 22 (158) (1927) 209–212.

[19] Q. McNemar, Note on the sampling error of the difference between correlated proportions or percentages, Psychometrika 12 (2) (1947) 153–157.

[20] M. López-Ibáñez, J. Branke, L. Paquete, Reproducibility in evolutionary computation, ACM Transactions on Evolutionary Learning and Optimization 1 (4) (2021) 1–21.

[21] T. Bartz-Beielstein, C. Doerr, D. van den Berg, J. Bossek, S. Chandrasekaran, T. Eftimov, A. Fischbach, P. Kerschke, W. La Cava, M. López-Ibáñez, K. M. Malan, J. H. Moore, B. Naujoks, P. Orzechowski, V. Volz, M. Wagner, T. Weise, Benchmarking in optimization: Best practice and open issues, arXiv:2007.03488 (2020).

[22] N. Hansen, A. Auger, R. Ros, O. Mersmann, T. Tušar, D. Brockhoff, COCO: a platform for comparing continuous optimizers in a black-box setting, Optimization Methods and Software 36 (1) (2021) 114–144.

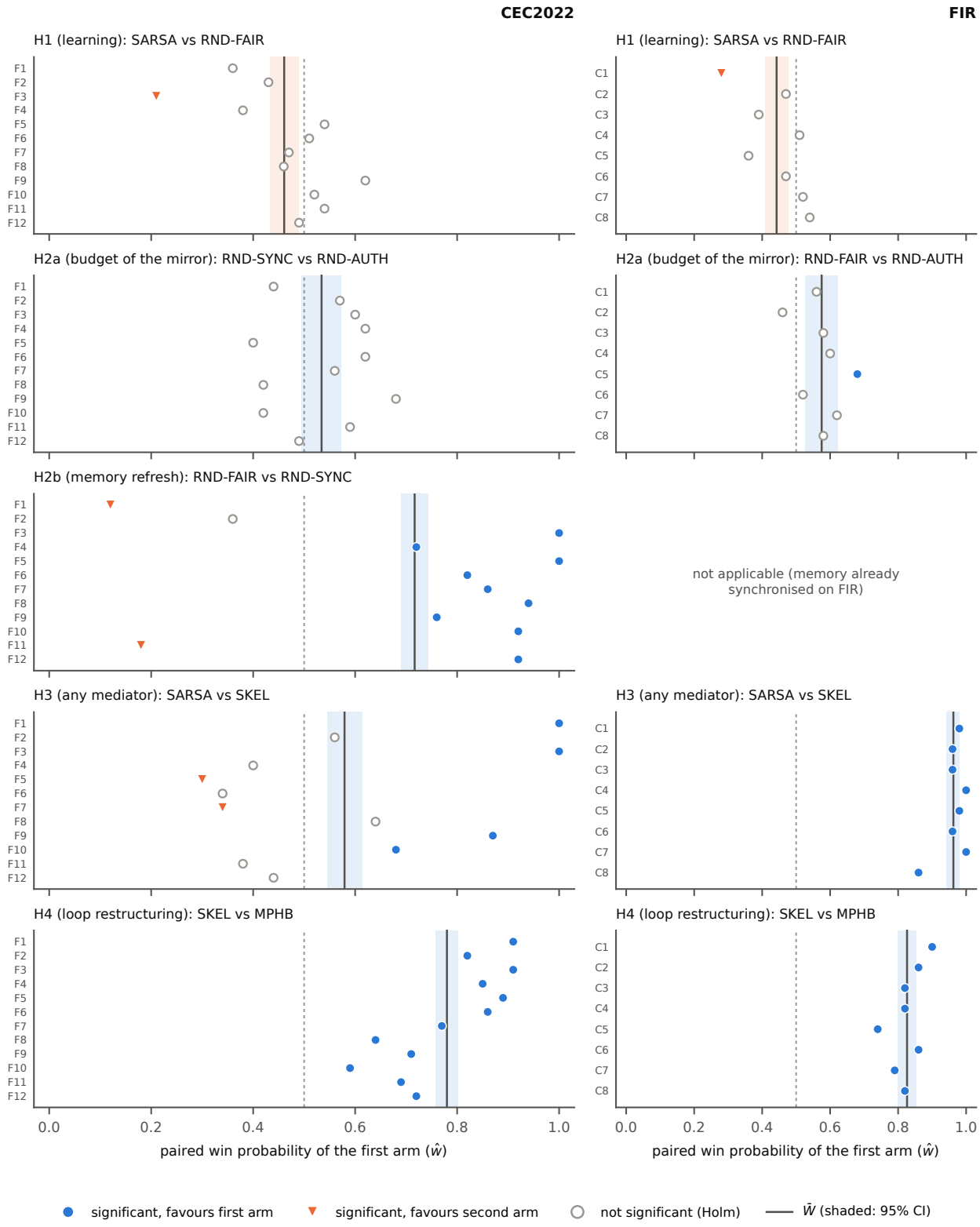


Figure 2: Per-instance paired win probability $\hat{w}$ of the first arm for the confirmatory hypotheses (F: CEC2022 function, C: FIR case). Filled markers are significant after Holm correction; the vertical line and band show $\bar{W}$ and its 95% bootstrap interval; the dotted line marks no difference.